

WHMCS Database Optimization Guide & Runbook

This guide contains detailed performance telemetry analysis, index tuning justification matrices, schema creation DDL, batch-pruning procedures, and application-level query guidelines. These optimizations directly address performance bottlenecks identified in the Elastic APM telemetry for the South Africa (TSA-WHMCS) and India (TIN-WHMCS) WHMCS shared database clusters.

1. Executive Summary & Latency Baselines

Elastic APM monitoring reveals distinct performance issues in each region. The India cluster is severely impacted by table lock contention during administrative session verification, while the South Africa cluster suffers from slow name-server queue checks, lock contention in log deletions, and filesort overhead on paginated invoice/client screens.

Regional Baseline Latency Comparison (Last 24 Hours)

Performance Metric	India WHMCS (TIN-WHMCS)	South Africa WHMCS (TSA-WHMCS)
Telemetry Source File	india-data.txt	sa-data.txt
Median Response Time (\$p50\$)	348.97 ms	467.29 ms
95th Percentile (\$p95\$)	5,123.49 ms	3,640.89 ms
99th Percentile (\$p99\$)	10,862.52 ms	11,068.87 ms
Primary System Bottleneck	tbladminlog Session Verification Lock Scans	DNSManager3_Job queues & tblactivitylog lock contention
Impacted Area	Admin Panel Actions (Up to 138s lag)	Addon Modules & Client Checkout Pages (Up to 33s lag)

2. Telemetry Deep-Dive: India (TIN-WHMCS)

The India cluster suffers from catastrophic slowdowns on administrative page views. Telemetry demonstrates that unindexed session checks on tbladminlog block write transactions, causing a ripple effect of thread locks across the database.

A. The Core Bottleneck: Unindexed Session Lookups

On every page view in the WHMCS admin panel, the application synchronously executes a session validation query:

```
SELECT id FROM tbladminlog WHERE sessionid = '?' AND lastvisit >= '?' AND logouttime = '0000-00-00 00:00:00' ORDER BY id DESC;
```

Because the tbladminlog table lacks an index on sessionid , MySQL executes a full table scan. In india-data.txt , there are 46 distinct executions of this query taking between 5.14s and 12.61s each.

- **Average Scan Duration:** ~7.9 seconds
- **Cumulative Impact:** ~363.4 seconds (6.05 minutes) of active thread-blocking database time.
- **Sample Telemetry Items:**
 - `sessionid = 'c6d3p4cf2fnrtb3k5a5kj0aed0'` (lastvisit >= 08:15:37) -> **12,606.85 ms**
 - `sessionid = 'c6d3p4cf2fnrtb3k5a5kj0aed0'` (lastvisit >= 08:16:34) -> **11,590.55 ms**
 - `sessionid = 'umun6qqn4t1f43rtk4asi2g41t'` (lastvisit >= 08:09:28) -> **10,617.01 ms**

B. Cascading Table Lock Contention

Because these unindexed `SELECT` scans run so slowly, they hold open InnoDB read locks and trigger lock waits on concurrent writes and updates to the session log table:

1. Slow Session Updates:

- `UPDATE tbladminlog SET logouttime = lastvisit WHERE adminusername = 'MosesA' AND lastvisit < ...` takes **5,949.23 ms** and **3,612.23 ms** per execution (lock wait timeout threshold).

2. Slow Session List Lookups:

- `SELECT * FROM tbladminlog WHERE lastvisit > ? GROUP BY adminusername ORDER BY lastvisit ASC` ran **14 times**, averaging **2,605.20 ms** per call (**36.47s cumulative**).

3. Slow Session Inserts:

- Normally instantaneous `INSERT INTO tbladminlog` commands take between **220.11 ms** and **430.16 ms** due to locking waits at the tail-end of session verification.

C. Downstream Impact on Admin Transactions

Because these database operations are synchronous and run on the primary PHP execution path, they cause massive response times for admin transactions:

- `POST /cloud/admin/clientsservices.php` -> **138,737.01 ms (138.7s)** (1 execution)
- `POST /cloud/admin/orders.php` -> **55,642.96 ms (55.6s)** (1 execution)
- `GET /cloud/admin/index.php` -> **29,413.82 ms (29.4s)** (2 executions)
- `GET /cloud/admin/billing/invoice/3497` -> **18,232.22 ms (18.2s)** (1 execution)
- `POST /cloud/admin/invoices.php` -> **9,455.80 ms (9.5s)** (23 executions, **217.48s cumulative**)

[!IMPORTANT] Resolution Priority: Creating a single index on `tbladminlog(sessionid)` is the highest-priority database action. It will immediately resolve these 10s+ page stalls and drop session lookups to < 2 ms .

3. Telemetry Deep-Dive: South Africa (TSA-WHMCS)

The South Africa cluster exhibits a more distributed query profile. Bottlenecks stem from log pruning write locks, slow custom addon queries, unindexed invoice joins, and expensive pagination filesorts.

A. Activity Log Table Locks (tblactivitylog)

A single clean-up query blocks concurrent events:

```
DELETE FROM tblactivitylog WHERE userid = ? AND id <= ?;
```

- **Average Duration: 6,244.49 ms (6.2s)**
- **Analysis:** This query locks index nodes during deletion, blocking concurrent `INSERT INTO tblactivitylog logs` (which spike to **227.31 ms**). Additionally, `SELECT COUNT(id) FROM tblactivitylog` takes **207.45 ms** to run, indicating table fragmentation.

B. Custom Addon DNS Manager Queue Scans (`DNSManager3_Job`)

The custom nameserver zone management module performs background checks on unindexed status tables:

1. `SELECT * FROM DNSManager3_Job WHERE job = ? AND status = ?`
 - **Average Duration: 843.64 ms**
 - **Execution Count: 21**
 - **Cumulative Impact: 17.72 seconds**
2. `SELECT DNSManager3_Job.* FROM DNSManager3_Job WHERE userid = ?`
 - **Average Duration: 518.57 ms** (1 execution)

C. Unindexed Invoice Item Verification Joins

During client checkout and portal invoice load, WHMCS runs verification checks across client and invoice items:

```
SELECT tblinvoices.id, tblinvoices.duedate FROM tblinvoices
INNER JOIN tblinvoiceitems ON tblinvoices.id = tblinvoiceitems.invoiceid
AND tblinvoiceitems.type = ? AND tblinvoiceitems.relid = ?
WHERE tblinvoices.status = ? ORDER BY tblinvoices.duedate ASC LIMIT 1;
```

- **Average Duration: 103.80 ms** (58 executions) and **107.21 ms** (35 executions)
- **Cumulative Impact: 9.77 seconds**
- **Analysis:** The table `tblinvoiceitems` lacks a composite index on `(type, relid, invoiceid)`, forcing full tablespace scans to verify item states.

D. Chronological and Alphabetical Filesorts (High-Offset Pagination)

Administrative export processes and lists scan large amounts of historic data using high offsets:

1. **Invoice Pagination:**
 - `SELECT ... FROM tblinvoices INNER JOIN tblclients ... ORDER BY tblinvoices.date DESC LIMIT 1000 OFFSET 20000 -> 221.48 ms`
 - `SELECT ... FROM tblinvoices INNER JOIN tblclients ... ORDER BY tblinvoices.date DESC LIMIT 1000 OFFSET 10000 -> 212.90 ms`
 - Thirty-five similar executions account for **~4.4 seconds** of cumulative database engine delay.
2. **Client Directory Pagination:**
 - `SELECT SQL_CALC_FOUND_ROWS ... FROM tblclients ORDER BY lastname ASC, firstname ASC ... LIMIT 9000, 1000 -> 147.54 ms`
 - Twelve similar pagination queries account for **~1.73 seconds** of cumulative sorting delay.

E. Domain Reminder Scans

Sequential checks are executed on unindexed fields during renewal notification crons:

```
SELECT * FROM tbldomainreminders WHERE domain_id = '?' ORDER BY id DESC;
```

- **Average Duration:** ~103.0 ms per call (5 sample entries in the top 50 slowest queries).

4. Telemetry-to-Index Justification Matrix

Target Table	SQL Query Shape	Telemetry Evidence (Duration / Count)	Proposed Index
tbladminlog	SELECT id FROM tbladminlog WHERE sessionid = ? AND lastvisit >= ?	5.8s – 12.6s avg latency (46 queries)	idx_tbladminlog_sessionid (sessionid)
DNSManager3_Job	select * from DNSManager3_Job where job = ? and status = ?	843.64 ms avg (21 executions, 17.7s cumulative)	idx_dnsmanager3_job_job_status (job, status)
DNSManager3_Job	select * from DNSManager3_Job where userid = ?	518.57 ms avg (1 execution)	idx_dnsmanager3_job_userid (userid)
tblinvoiceitems	select duedate from tblinvoices inner join tblinvoiceitems...	103ms – 107ms avg (93 executions, 9.77s cumulative)	idx_tblinvoiceitems_type_relid_invoiceid
tblinvoices	order by tblinvoices.date desc limit 1000 offset 20000	221.48 ms avg (35 executions, 4.4s cumulative)	idx_tblinvoices_date (date)
tblclients	ORDER BY lastname ASC, firstname ASC... LIMIT 9000, 1000	138ms – 147ms avg (13 executions, 1.8s cumulative)	idx_tblclients_name_search

tbldomainreminders	SELECT * FROM tbldomainreminders WHERE domain_id = ?	~103 ms avg (5 executions in top slow queries)	idx_tbldomainreminders_domain_id (domain_id)
---------------------------	---	---	---

5. Capture a Baseline & Schema Inspect

Run these query commands before applying any DDL changes to confirm existing schema state, verify sizing limits, and review InnoDB buffer pools.

```
-- Check current index state
SHOW CREATE TABLE tbladminlog;
SHOW CREATE TABLE DNSManager3_Job;
SHOW CREATE TABLE tbldomainreminders;
SHOW CREATE TABLE tblinvoices;
SHOW CREATE TABLE tblclients;
SHOW CREATE TABLE tblinvoiceitems;

-- Review row counts and database footprint sizes
SELECT table_name, table_rows,
       ROUND(data_length/1024/1024,1) AS data_mb,
       ROUND(index_length/1024/1024,1) AS index_mb
FROM information_schema.tables
WHERE table_schema = DATABASE()
      AND table_name IN ('tbladminlog', 'tblactivitylog', 'DNSManager3_Job',
                        'tbldomainreminders', 'tblinvoices', 'tblclients', 'tblinvoiceitems');

-- Check InnoDB buffer pool allocation (target is ~60-70% of database RAM)
SHOW VARIABLES LIKE 'innodb_buffer_pool_size';
```

6. DDL Index Creation Statements

Apply these DDL queries during a low-traffic maintenance window. The `ALGORITHM=INPLACE` and `LOCK=NONE` parameters ensure these tables remain readable during index builds.

```
-- 1. Admin Session Verification (Critical for India Deployment)
-- Satisfies high-latency session checks (avg 5.8s - 12.6s)
ALTER TABLE tbladminlog
  ADD INDEX idx_tbladminlog_sessionid (sessionid),
  ALGORITHM=INPLACE, LOCK=NONE;

-- 2. DNSManager3 Job Queue (High Priority for South Africa)
-- Satisfies custom nameserver status checks (avg 518ms - 843ms)
ALTER TABLE DNSManager3_Job
  ADD INDEX idx_dnsmanager3_job_job_status (job, status),
  ADD INDEX idx_dnsmanager3_job_userid (userid),
  ALGORITHM=INPLACE, LOCK=NONE;
```

```

-- 3. Domain Renewal Reminders
-- Satisfies cron check lookups (avg 103ms)
ALTER TABLE tbldomainreminders
  ADD INDEX idx_tbldomainreminders_domain_id (domain_id),
  ALGORITHM=INPLACE, LOCK=NONE;

-- 4. Invoice Items Lookups
-- Satisfies checkout item inner joins (avg 103ms - 107ms)
CREATE INDEX idx_tblinvoiceitems_type_relid_invoiceid ON tblinvoiceitems (type,
relid, invoiceid);
CREATE INDEX idx_tblinvoiceitems_type_relid ON tblinvoiceitems (type, relid);

-- 5. Invoice Date Sorting
-- Resolves offset pagination filesort locks (avg 221ms)
ALTER TABLE tblinvoices
  ADD INDEX idx_tblinvoices_date (date),
  ALGORITHM=INPLACE, LOCK=NONE;

-- 6. Client Admin Dashboards & Search
-- Optimizes paging and sorting clients alphabetically by name (avg 147ms)
CREATE INDEX idx_tblclients_name_search ON tblclients (lastname(64), firstname(64),
companyname(64));

```

*[!NOTE] **Duplicate Index Cleanup:** During the baseline inspection, check if `tblactivitylog` has duplicate indexes on both `userid` and `user_id`. Having both increases index maintenance overhead on every insert/delete. If confirmed duplicate, drop the redundant index:*

```
ALTER TABLE tblactivitylog DROP INDEX user_id, ALGORITHM=INPLACE, LOCK=NONE;
```

7. Log Table Batch-Pruning (Safe Initial Cleanup)

Do not run a single large `DELETE` query on `tblactivitylog` (which locked the table for **6.2 seconds** in the APM log). Instead, run deletions in smaller batches of **5,000 rows** to avoid locking out concurrent logging requests.

Option A: Bash Batch Deletion Script

Execute this from the application shell or DB node:

```

while true; do
  AFFECTED=$(mysql -N -e "DELETE FROM tblactivitylog WHERE date <= DATE_SUB(NOW(),
INTERVAL 90 DAY) LIMIT 5000; SELECT ROW_COUNT();")
  echo "Deleted activity logs: $AFFECTED"
  if [ "$AFFECTED" -eq 0 ]; then
    echo "Pruning complete."
    break;
  fi
  sleep 1 # Pause briefly to prevent I/O saturation and allow concurrent locks to

```

```
clear
done
```

Option B: Native MySQL Event Scheduler

If Event Scheduling is active on the MySQL node, establish a daily maintenance event:

```
CREATE EVENT IF NOT EXISTS prune_tblactivitylog
ON SCHEDULE EVERY 1 DAY STARTS '2026-06-26 02:00:00'
DO
    DELETE FROM tblactivitylog WHERE date <= (CURDATE() - INTERVAL 90 DAY) LIMIT 5000;
```

8. Defragmentation & Optimizer Statistics

Following bulk deletions or schema updates, defragment the index trees to reclaim disk blocks and update statistical tables for the MySQL Optimizer:

```
-- Update statistics
ANALYZE TABLE tblinvoiceitems, tblactivitylog, DNSManager3_Job, tblclients,
tbldomainreminders, tblinvoices, tbladminlog;

-- Rebuild tables and defragment indexes (Run ONLY during off-peak hours)
OPTIMIZE TABLE tblinvoiceitems, tblactivitylog, DNSManager3_Job, tblclients,
tbldomainreminders, tblinvoices, tbladminlog;
```

9. Application-Level Queries Optimization

Provide these recommendations to the development team to optimize how PHP queries the database:

A. Batch N+1 Lookups (Looping Queries)

Avoid executing a SQL query inside a loop (e.g. looking up domain reminders per domain). Batch them into a single `WHERE IN` statement:

- **Before:**

```
-- Executed 50 times in a loop:
SELECT * FROM tbldomainreminders WHERE domain_id = ?;
```

- **After (Single Query):**

```
SELECT * FROM tbldomainreminders WHERE domain_id IN (12, 14, 15, 23, ...)
ORDER BY domain_id, id DESC;
```

Group the results in PHP memory after fetching.

B. Keyset Pagination (Avoid `OFFSET`)

WHMCS export scripts use high offset values (e.g., `LIMIT 1000 OFFSET 20000`). This forces MySQL to scan and discard 20,000 rows. Use **Keyset/Cursor Pagination** instead:

- **Before:**

```
SELECT ... FROM tblinvoices ORDER BY tblinvoices.date DESC LIMIT 1000 OFFSET 20000;
```

- **After (Cursor-based):**

```
SELECT ... FROM tblinvoices WHERE tblinvoices.id < :last_seen_id ORDER BY tblinvoices.id DESC LIMIT 1000;
```

Feed the minimum ID from the previous page into the next page request.

C. Remove `SQL_CALC_FOUND_ROWS`

Some listing pages run `SQL_CALC_FOUND_ROWS` on queries to get the total row count. This modifier is deprecated in MySQL 8.0 and forces a full scan of the dataset. Run a separate, cached `SELECT COUNT(*)` query only when the total count is explicitly required by the UI.

D. Static Array / Application Query Caching

The shared database cluster is under heavy pressure from high-frequency queries fetching static configuration and theme data:

- **tblconfiguration** (81,155 queries/day) & **tblcurrencies** (35,570 queries/day)
- **Lagom Theme menus** (`rsthemes_menus_content` & `rstheme_themes` — 106,000+ queries/day)
- *Solution:* Implement static array caching in PHP memory (or use a shared Redis cache) so that configuration and menu queries are executed **once per request** instead of inside N+1 loops. This will instantly eliminate **over 220,000 redundant SELECT queries per day** from the database cluster.

E. Eliminate Redundant External API Debug Calls

Telemetry flags active outbound HTTP calls to `httpbin.org` (Average duration: 10.0 seconds, 36 executions, Cumulative: 360 seconds). This is a debug/testing endpoint that was accidentally left in a plugin or hook file. Because it runs synchronously, it blocks PHP execution threads for 10 seconds. Audit hooks and modules to locate and disable this test call.

10. Target Milestones

Compare pre- and post-optimization execution times:

Query Shape / Target	Pre-Optimization Latency	Post-Index / Script Target
tbladminlog session lookup	5.0s to 12.6s	< 2 ms (via sessionid index)
DNSManager3_Job status lookup	~843 ms	< 10 ms
DNSManager3_Job user lookup	~518 ms	< 10 ms

tbldomainreminders lookup	~103 ms	< 10 ms
tblclients admin search	~147 ms	< 15 ms
tblactivitylog delete lock time	6.2 seconds	< 50 ms (via 5k limit)